

gemini-conversation

Exported on: 7/19/2026, 7:07:25 PM

User

i want a math explanation of the apes algorithm

Gemini

The **APES (Amplitude and Phase Estimation)** algorithm is a powerful, non-parametric, data-adaptive filter bank method used in signal processing and spectral analysis. It was introduced by Jian Li and Petre Stoica in 1996.

MetricGate + 1

While a traditional Fast Fourier Transform (FFT) uses fixed sinusoidal filters (which can lead to leakage and blurred peaks if data records are short), APES designs a **custom, optimal filter for every single frequency** based on the signal itself.

MetricGate

Here is the exact mathematical formulation of how it works.

1. The Signal Model

Suppose you have a discrete-time signal $y(t)$ measured at N time steps: $t = 0, 1, \dots, N - 1$. At any specific frequency of interest ω , we model the signal as a single complex sinusoid plus noise:

$$y(t) = \alpha(\omega)e^{j\omega t} + e_{\omega}(t)$$

Where:

- $\alpha(\omega)$ is the **complex amplitude** at frequency ω . The absolute value $|\alpha(\omega)|$ gives you the true peak amplitude, and the angle $\arg(\alpha(\omega))$ gives you the exact phase.

MetricGate

- $e_{\omega}(t)$ is the unmodeled residual noise (which includes noise plus components from all other frequencies).

2. Snapshot Formulation

To solve for $\alpha(\omega)$ adaptively, APES uses a sliding window of length M (where $M < N$). We extract $L = N - M + 1$ overlapping segments called **snapshots**:

MetricGate

$$\mathbf{y}_l = [y(l) \quad y(l+1) \quad \dots \quad y(l+M-1)]^T, \quad l = 0, 1, \dots, L-1$$

Using this windowing technique, our signal model can be rewritten in vector form for each snapshot l :

$$\mathbf{y}_l = \alpha(\omega)e^{j\omega l}\mathbf{a}(\omega) + \mathbf{e}_l(\omega)$$

Where $\mathbf{a}(\omega)$ is the **Fourier steering vector** of length M :

$$\mathbf{a}(\omega) = [1 \quad e^{j\omega} \quad e^{j2\omega} \quad \dots \quad e^{j(M-1)\omega}]^T$$

3. The Optimization Problem

To find $\alpha(\omega)$, APES designs a Finite Impulse Response (FIR) filter vector $\mathbf{h}(\omega)$ of length M . The filter is constrained to pass the frequency ω perfectly with a gain of 1:

$$\mathbf{h}^H(\omega)\mathbf{a}(\omega) = 1$$

(where H denotes the conjugate transpose).

ResearchGate

Unlike the classic Capon (MVDR) method—which minimizes total output power—APES is a **least-squares match**. It minimizes the mean squared error between the filtered signal output and the expected filtered sinusoid:

ResearchGate

$$\min_{\mathbf{h}, \alpha} \frac{1}{L} \sum_{l=0}^{L-1} |\mathbf{h}^H \mathbf{y}_l - \alpha e^{j\omega l}|^2 \quad \text{subject to} \quad \mathbf{h}^H \mathbf{a}(\omega) = 1$$

4. Solving the Math

By holding $\mathbf{h}$ constant, we can take the derivative with respect to α to find its optimal value. This yields:

$$\alpha(\omega) = \mathbf{h}^H(\omega)\mathbf{g}(\omega)$$

Where $\mathbf{g}(\omega)$ is the normalized Fourier transform of the snapshots (often called the **cross-spectral vector**):

$$\mathbf{g}(\omega) = \frac{1}{L} \sum_{l=0}^{L-1} \mathbf{y}_l e^{-j\omega l}$$

Substituting $\alpha(\omega)$ back into the optimization objective transforms the problem into finding the filter vector $\mathbf{h}$ that minimizes a modified covariance matrix:

$$\min_{\mathbf{h}} \mathbf{h}^H \hat{\mathbf{Q}}(\omega) \mathbf{h} \quad \text{subject to} \quad \mathbf{h}^H \mathbf{a}(\omega) = 1$$

Where $\hat{\mathbf{Q}}(\omega)$ is the estimated **noise-plus-interference covariance matrix**:

$$\hat{\mathbf{Q}}(\omega) = \hat{\mathbf{R}} - \mathbf{g}(\omega)\mathbf{g}^H(\omega)$$

Here, $\hat{\mathbf{R}}$ is the standard sample covariance matrix of the signal snapshots:

$$\hat{\mathbf{R}} = \frac{1}{L} \sum_{l=0}^{L-1} \mathbf{y}_l \mathbf{y}_l^H$$

5. The Final APES Estimate

Using Lagrange multipliers to solve the constrained optimization for $\mathbf{h}$, we find the optimal filter:

$$\mathbf{h}(\omega) = \frac{\hat{\mathbf{Q}}^{-1}(\omega) \mathbf{a}(\omega)}{\mathbf{a}^H(\omega) \hat{\mathbf{Q}}^{-1}(\omega) \mathbf{a}(\omega)}$$

Plugging this optimal filter back into our expression for $\alpha(\omega)$ gives the final, elegant APES equation for complex amplitude:

$$\alpha(\omega) = \frac{\mathbf{a}^H(\omega) \hat{\mathbf{Q}}^{-1}(\omega) \mathbf{g}(\omega)}{\mathbf{a}^H(\omega) \hat{\mathbf{Q}}^{-1}(\omega) \mathbf{a}(\omega)}$$

Why go through all this trouble? (APES vs. Capon)

If you are familiar with the Capon MVDR spectral estimator, its power spectrum formula looks like this:

$$P_{\text{Capon}}(\omega) = \frac{1}{\mathbf{a}^H(\omega) \hat{\mathbf{R}}^{-1} \mathbf{a}(\omega)}$$

Notice that Capon uses the raw covariance matrix $\hat{\mathbf{R}}^{-1}$, meaning it treats the target signal at ω as part of the total energy it is trying to minimize. Because of this, **Capon systematically underestimates (attenuates) the true peak amplitudes.**

By subtracting the cross-spectral vector ($\hat{\mathbf{Q}} = \hat{\mathbf{R}} - \mathbf{g}\mathbf{g}^H$), APES explicitly removes the target frequency's energy from the minimization process. As a result, it filters out the noise perfectly without shrinking or biasing your true signal peaks.

User

given a signal stream show me how one runs the traditional fft and then runs the apes

Gemini

To see how APES behaves compared to a traditional Fast Fourier Transform (FFT), let's walk through a concrete data example.

The biggest conceptual jump between the two is that an **FFT handles the entire signal block globally at once**, whereas **APES operates snapshot-by-snapshot and frequency-by-frequency** via an optimization problem.

Here is exactly how you would process a stream of N data points using both methods.

1. The Setup: The Data Stream

Imagine your receiver is pulling a discrete-time signal stream of $N = 64$ samples. Inside this stream is a real target signal at a specific frequency (let's say $\omega_0 = 0.25 \times 2\pi$ rad/sample), buried under some noise and a massive interfering signal at a nearby frequency.

We store this incoming stream as a simple vector:

$$\mathbf{y}_{stream} = [y(0) \quad y(1) \quad \dots \quad y(63)]^T$$

We want to estimate the spectrum over a grid of K discrete frequencies (e.g., $K = 128$ points between 0 and 2π). Let's pick an evaluation frequency ω from our grid and pass our stream through both algorithms.

2. Running the Traditional FFT

The FFT is straightforward. It maps the N time-domain points directly to N orthogonal frequency bins.

Step A: Windowing (Optional but standard)

To mitigate spectral leakage, you multiply your raw stream element-wise by a passive window function w (like a Hamming or Hann window):

$$y_{windowed}(t) = y(t) \cdot w(t)$$

Step B: The Core Fast Fourier Transform

You run the Radix-2 FFT algorithm on the vector. Mathematically, for our specific chosen frequency ω , this computes a simple projection (discrete Fourier transform):

$$Y_{FFT}(\omega) = \sum_{t=0}^{N-1} y_{windowed}(t) e^{-j\omega t}$$

Step C: Amplitude Scaling

To turn this raw output into a true physical amplitude estimate, you must divide by the coherent gain of your window function (for a rectangular window, this is just N):

$$\alpha_{FFT}(\omega) = \frac{1}{N} Y_{FFT}(\omega)$$

- **Execution Speed:** Fast ($O(N \log N)$).
- **The Catch:** If the huge interfering signal is close to our target, its energy leaks through the window sidelobes, blurring the two peaks together or burying the target completely.

3. Running the Adaptive APES Algorithm

Instead of a global transform, APES builds a custom adaptive filter for our chosen frequency ω . Because it needs to estimate a covariance matrix, we first choose a filter length (window size) M . A common rule of thumb is $M = N/3$ or $M = N/2$. Let's choose $M = 24$.

Here is the exact algorithmic loop you execute for APES:

Step A: Slice the Stream into Snapshots

We slide a window of size $M = 24$ across our $N = 64$ data stream. This creates $L = N - M + 1 = 41$ overlapping vector snapshots:

$$\mathbf{y}_0 = \begin{bmatrix} y(0) \\ \vdots \\ y(23) \end{bmatrix}, \quad \mathbf{y}_1 = \begin{bmatrix} y(1) \\ \vdots \\ y(24) \end{bmatrix}, \quad \dots, \quad \mathbf{y}_{40} = \begin{bmatrix} y(40) \\ \vdots \\ y(63) \end{bmatrix}$$

Step B: Compute the Global Sample Covariance

We compute how the data inside these windows varies across the snapshots to build our $M \times M$ matrix $\hat{\mathbf{R}}$:

$$\hat{\mathbf{R}} = \frac{1}{41} \sum_{l=0}^{40} \mathbf{y}_l \mathbf{y}_l^H$$

Step C: Enter the Frequency Loop

Now, for our specific evaluation frequency ω , we execute the adaptive magic.

1. **Generate the Steering Vector:** Create the ideal phase progression vector $\mathbf{a}(\omega)$ of length M :

$$\mathbf{a}(\omega) = [1 \quad e^{j\omega} \quad e^{j2\omega} \quad \dots \quad e^{j23\omega}]^T$$

2. **Compute the Cross-Spectral Vector:** Find the localized Fourier transform of our snapshots at this frequency:

$$\mathbf{g}(\omega) = \frac{1}{41} \sum_{l=0}^{40} \mathbf{y}_l e^{-j\omega l}$$

3. **Subtract the Target Energy:** Strip the energy of ω out of our covariance matrix to form the noise-plus-interference matrix $\hat{\mathbf{Q}}(\omega)$:

$$\hat{\mathbf{Q}}(\omega) = \hat{\mathbf{R}} - \mathbf{g}(\omega)\mathbf{g}^H(\omega)$$

4. **Invert and Solve:** Invert $\hat{\mathbf{Q}}(\omega)$ (or use a stable linear solver like Cholesky/QR decomposition) to find the amplitude:

$$\alpha_{APES}(\omega) = \frac{\mathbf{a}^H(\omega)\hat{\mathbf{Q}}^{-1}(\omega)\mathbf{g}(\omega)}{\mathbf{a}^H(\omega)\hat{\mathbf{Q}}^{-1}(\omega)\mathbf{a}(\omega)}$$

Step D: Repeat For All Frequencies

You repeat **Step C** for every single frequency ω in your grid. Because $\mathbf{g}(\omega)$ and $\hat{\mathbf{Q}}(\omega)$ depend explicitly on ω , you must recompute them at every step.

Architectural Comparison Matrix

Data Chunking	Processes all N points in a single block.	Slides a smaller window M to create L snapshots.
Data Dependency	Fixed basis vectors ($e^{-j\omega t}$). Blind to signal content.	Data-adaptive. Calculates $\hat{\mathbf{R}}$ from the signal structure.

Frequency Handling	All bins are calculated simultaneously in one pass.	Every frequency grid point requires its own matrix adjustments.	
Computational Cost	Extremely low ($O(N \log N)$).	Extremely high ($O(K \cdot M^3)$) due to per-frequency matrix inversions.	
The Result	Fast, but suffers from low resolution and high sidelobe leakage.	Sharp, highly resolved peaks with perfectly preserved amplitudes.	
...			